

Projeto e Análise de Algoritmos

Trabalho 1 – Algoritmo da árvore de Huffman

Professor: Rômulo César Silva.

Nome: Marcos Oliveira de Sousa.

Legenda:

1. **MB:** megabyte.
2. **ms:** milissegundo.
3. **PDF:** Portable Document Format, extensão para documentos.
4. **TXT:** extensão para arquivo de texto.

Contextualização

Para compactação foram criados três arquivos de textos com tamanhos diferentes, um arquivo com 31MB, 182.1MB e 109MB com 479,632 linhas, 3,343,882 linhas e 1,977,632 linhas respectivamente. O conteúdo dos três arquivos contém livros de diferentes autores como Tanenbaum, Presmann, Sommerville, Proust além de slides de aulas convertidos de PDF para TXT.

O algoritmo desenvolvido é capaz de receber um texto com quaisquer símbolos contidos, codificar e comprimir em um novo arquivo binário, bem como é possível decodificar e tornar no texto original. Para tal é criado um cabeçalho no arquivo binário com o número de símbolos, para cada símbolo o tamanho correspondente, a string (em caractere ou em palavra) e a frequência para a criação de uma árvore de *huffman*.

Segue abaixo duas tabelas que demonstram a eficiência na compressão, tanto em tempo medido, quanto em tamanho comprimido, também foi calculado dois percentuais, um para demonstrar a diferença entre o tamanho comprimido em relação ao tamanho total e outro para demonstrar a diferença entre o tempo de compressão e descompressão. Para garantir que a integridade dos dados fora mantida, foi optado por adicionar uma linha contendo o tamanho do arquivo após a descompressão.

Compactação por caractere			
XXXXXXXXXXXXXXXXXXXX	Arquivo 1	Arquivo 2	Arquivo 3
Tamanho inicial	30,501,715 bytes	182,128,542 bytes	109,045,481 bytes
Tempo de compressão	22,119 ms	152,094 ms	76,431 ms
Tamanho comprimido	17,055,795 bytes	107,091,893 bytes	63,992,656 bytes
Razão da compressão	55.92%	58.8%	58.68%
Tempo de descompressão	3,471 ms	21,380 ms	12,772 ms
Razão de tempo	15.69%	14.06%	16.71%
Tamanho descomprimido	30,501,715 bytes	182,128,542 bytes	109,045,481 bytes

Tabela 1. Compactação por caractere em três arquivos de diferentes tamanhos (Fonte: o autor).

Compactação por palavra			
XXXXXXXXXXXXXXXXXXXX	Arquivo 1	Arquivo 2	Arquivo 3
Tamanho inicial	30,501,715 bytes	182,128,542 bytes	109,045,481 bytes
Tempo de compressão	26,865 ms	154,358 ms	88,988 ms
Tamanho comprimido	12,589,434 bytes	71,880,210 bytes	43,396,583 bytes
Razão da compressão	41.27%	39.47%	39.8%
Tempo de descompressão	2,365 ms	12,634 ms	7,723 ms
Razão de tempo	8.8%	8.18%	8.68%
Tamanho descomprimido	30,501,715 bytes	182,128,542 bytes	109,045,481 bytes

Tabela 2. Compactação por palavra em três arquivos de diferentes tamanhos (Fonte: o autor).

Estruturas de dados

Segue abaixo as estruturas de dados diretamente utilizadas:

1. Map

Armazena elementos em pares chave-valor, são armazenadas de maneira ordenada, a busca de um elemento tende a ser rápido.

```
#include <map>
```

2. Fila de prioridade

Mantém uma coleção de elementos, cada um com uma prioridade associada, para o programa, a prioridade associada é a frequência de cada caractere/palavra.

```
#include <queue>
```

3. Vetor

Estrutura para guardar elementos de forma contígua na memória com índice associado a cada elemento.

```
#include <vector>
```

4. Estrutura de comparação para fila de prioridade

Estrutura criada para comparar dois nós do tipo árvore huffman para retornar a prioridade do par de nós.

```
struct comparadorNo
```

Manipulação de bits

A linguagem C++ (bem como a linguagem C) permite fácil manipulação de bits, tanto para rotacionar bits, comparar, aplicar operadores (AND, NOT, XOR).

Segue abaixo alguns operadores em C/C++ para manipulação de bits:

Descrição do operador	Operador
Mover bits para a esquerda	<<
Mover bits para a direita	>>
NOT	!
AND	&
OR	
XOR	^

Tabela 3. Operadores para manipulação de bits (Fonte: o autor).

Os operadores usados nesse trabalho foram de movimentação de bits, AND e OR.

Medição de tempo

Para medir o tempo foi considerado apenas o tempo de compressão e descompressão, desprezando o tempo que o usuário possa tomar no menu de seleção. Para isso foi utilizado a biblioteca a seguir:

```
include <chrono>
```